

Work Division

Component	Who	Status
Program Counter (pc_reg)	Jian	Started (basic Implementation done)
PC and target addr (adder16)	Jian	
Instruction Mem (instr_mem)	Jordan	
Data Mem (data_mem)	Jordan	
Sign Ext (sign_extend)	Jordan	
Branch/Jump (branch_logic)	Jordan	
Datapath Diagram	Jordan	
Register File (reg_file)	Jian	
ALU (alu)	Jordan	Started (basic Implementation done)
ALU testbench (alu_tb)	Jordan	Started (basic Implementation done)
ALU Control (alu_control)	Jordan	
Control Unit (control_unit)	Jian	
Multiplexers (mux2)	Jian	
Control Table	Jian	
Top Level Datapath (cpu_top)	Both	
Full CPU testbench (cpu_tb)	Both	

(we will also be writing test benches for each of the assigned models. Later on for the demo, Jordan will build buttons for the debouncer and the 7 seg driver, and Jian will build the Basys 3 wrapper and constraints.)

Pledge

We agree to this division of work and each of us pledge to contribute to the best of our abilities while keeping the other updated on progress, and ask for help early on if we fall behind.

Person 1 Signature: Jordan Korogodsky

Date: 10/04/2026

Person 2 Signature: Jian Xi Lu

Date: 10/4/2026

Weekly Log

Meetings

Date	Length	Topics
09/29/2026	1 hr	Splitting the work and project planning
10/01/2026	2.5 hrs	Splitting the work and project planning
10/04/2026	2 hrs	Finalizing the checkpoint 1 submission

Work Progress by Week

Week	Jordan	Jian
09/28/2026	Started draft of work division plan. Installed software. Finalized work division draft and started weekly log. Checked WorkDivision, WeeklyLog, SourceCode, and ProjectReport and prepared them for Checkpoint 1.	Installed software and began work on code. Finalized work division draft and started weekly log. Checked WorkDivision, WeeklyLog, SourceCode, and ProjectReport and prepared them for Checkpoint 1.
10/05/2026		
10/12/2026		
10/19/2026		
10/26/2026		

Code

ALU

```
module alu(  
    input [15:0] rs,  
    input [15:0] r,  
    input [1:0] op,  
    output reg [15:0] res  
);  
  
always @(*) begin  
    if(op == 2'b00)  
        res = rs + r;  
    else if(op == 2'b01)  
        res = rs - r;  
    else if(op == 2'b10)  
        res = rs << r;  
    else  
        res = rs & r;  
end  
endmodule
```

ALU Test Benchmark

```
module alu_tb;  
    reg [15:0] rs;  
    reg [15:0] r;  
    reg [1:0] op;  
    wire [15:0] res;  
  
    alu dut(.rs(rs), .r(r), .op(op), .res(res));  
  
    intial begin  
        a=5; b=3; op=2'b00;  
        #10;  
        $display("add: %0d + %0d = %0d", rs, r, res);  
        //expected out = 8  
  
        a=10; b=3; op=2'b01;  
        #10;
```

```

$display("sub: %0d + %0d = %0d", rs, r, res);
    //expected out = 7

    a=4; b=90; op=2'b10;
#10;
    $display("add: %0d + %0d = %0d", rs, r, res);
    //expected out = 1440

    a=12; b=10; op=2'b11;
#10;
    $display("add: %0d + %0d = %0d", rs, r, res);
    //expected out = 8

    $finish;
end
endmodule

```

Program Counter

```

module Program_counter(
    input clock,
    input reset,
    input [15:0] next,
    output reg [15:0] out
);
initial
begin
    out=16'b0000000000000000;
end

always @(posedge clock)begin
    if (reset==1'b1)
        out<=16'b0000000000000000;
    else
        out<=next;
end

endmodule

```